Unidad 3: Invocación Remota y Contenerización

1. Modelo Cliente-Servidor y Middleware

El modelo cliente-servidor es la base de la computación distribuida: un cliente realiza pedidos y un servidor los procesa. La comunicación más básica se realiza a través de **Sockets**, que proveen una interfaz de lectura/escritura sobre la red. Sin embargo, los sockets presentan limitaciones:

- Obligan a trabajar con mecanismos de E/S de bajo nivel.
- La programación resulta más compleja que con una interfaz funcional.
- La E/S directa no es el camino ideal cuando se quiere que la computación distribuida se asemeje a la computación centralizada.

Para resolver estos problemas nace el **Middleware**: una capa de software intermedia que provee servicios y protocolos reutilizables por múltiples aplicaciones. Sus responsabilidades incluyen:

- Protocolos de comunicación ricos (marshaling/unmarshaling de datos).
- Protocolos de nombrado para compartir recursos fácilmente.
- Protocolos de seguridad para comunicaciones seguras.
- Mecanismos de escalabilidad (replicación, caché).

2. Tipos de Comunicación

La comunicación entre procesos puede clasificarse en dos ejes:

Sincrónica vs. Asincrónica

- En la comunicación sincrónica, el emisor queda bloqueado hasta recibir la respuesta.
- En la asincrónica, el emisor continúa su ejecución sin esperar la respuesta.

Transiente vs. Persistente

- **Transiente/Transitoria**: el servidor descarta el mensaje si no puede entregarlo de inmediato al receptor.
- **Persistente**: el mensaje se almacena en el servidor de comunicación hasta que pueda ser entregado.

Los protocolos de intercambio más conocidos (identificados por Alfred Spector, 1982) son:

- **R (Request)**: solo se envía la solicitud.
- **RR (Request-Reply)**: solicitud y respuesta.
- **RRA (Request-Reply-Acknowledge Reply)**: solicitud, respuesta y confirmación de recepción.

3. Invocación Remota (RMI)

El término **RMI (Remote Method Invocation)** se usa de forma genérica para referirse a la invocación de un método en un proceso remoto. Extiende dos modelos:

- El modelo de **llamada a procedimiento remoto (RPC)**, propio de la programación estructurada.
- El modelo de **programación orientada a objetos (POO)**, permitiendo invocar métodos de objetos que residen en otra máquina.

El objetivo es que la llamada remota sea lo más transparente posible respecto a una llamada local.

4. Semántica de Invocación

Dado que una llamada remota puede fallar de maneras que una llamada local no puede, se definen distintas garantías de ejecución:

Semántica "Pudiera Ser" (*Maybe*)

- El invocante no puede determinar si el método se ejecutó una vez o ninguna.
- Aparece cuando **no se aplica ninguna medida de tolerancia a fallos**.
- Tipos de fallo posibles: omisión (pérdida del mensaje) o caída del servidor.
- Si no hay respuesta tras un timeout y no se realizan reintentos, no hay certeza de ejecución.

Semántica "Al Menos Una Vez" (*At Least Once*)

- El invocante recibe un resultado o una excepción; si recibe resultado, el método se ejecutó **al menos una vez**.
- El mensaje de invocación puede retransmitirse, lo que podría causar que el método se ejecute **más de una vez**.
- Requiere el uso de **operaciones idempotentes** (que pueden ejecutarse repetidamente sin efectos distintos).
- Sun RPC implementa esta semántica.

Semántica "Como Máximo Una Vez" (*At Most Once*)

- El invocante recibe un resultado o una excepción; si recibe resultado, el método se ejecutó **exactamente una vez**.
- Se obtiene aplicando todas las medidas de tolerancia a fallos disponibles.
- Implementada por **Java RMI y CORBA**. CORBA permite la semántica "pudiera ser" para métodos que no devuelven resultados.

5. Codificación de Datos para el Intercambio

Cualquier mecanismo de invocación remota necesita transmitir datos a través de la red como una secuencia de bytes. Para ello, emisor y receptor deben acordar un mecanismo de codificación. Los aspectos clave son:

- Qué tipos de datos se utilizarán (enteros, caracteres, reales, etc.).
- El orden en que se enviarán.
- El juego de caracteres a emplear.

Representación Binaria: Big-Endian y Little-Endian

El **endianness** define el orden en que se almacenan los bytes de un dato de múltiples bytes:

- **Little-Endian**: el byte menos significativo se guarda en la dirección de memoria menor. Ventaja: las operaciones matemáticas son más sencillas. Ejemplos: procesadores Pentium, Windows.
- **Big-Endian**: el byte más significativo se guarda en la dirección de memoria menor. Ventaja: permite saber rápidamente si un número es positivo o negativo; los datos se guardan tal como se escriben. Ejemplos: procesadores Motorola, Macintosh.

Codificación Textual vs. Binaria

- **Textual**: todo se convierte a texto; muy legible y fácil de depurar, pero ineficiente en espacio y velocidad.
- **Binaria**: no busca legibilidad sino eficiencia en espacio y procesamiento.

Delimitación de Datos

- **Longitud fija**: compacta pero inflexible (el efecto año 2000 fue consecuencia de este tipo).
- **Por longitud**: el emisor antepone el tamaño del dato; requiere recorrer el dato antes de enviarlo.
- **Delimitada**: se reserva un símbolo separador (por ejemplo, ";").

Marshalling y Unmarshalling

- **Marshalling (serialización)**: proceso de transformar la representación en memoria de un objeto a un formato apto para su transmisión o almacenamiento.
- **Unmarshalling**: proceso inverso; reconstruye las estructuras de datos a partir de la representación externa recibida.

6. Formatos de Codificación

XDR — External Data Representation

- Soporta todos los tipos del lenguaje C.
- Define una forma canónica intermedia (big-endian).
- No usa etiquetas de tipo (son implícitas).
- Usa enteros de longitud fija; todos los datos ocupan múltiplos de 4 bytes.
- Ventaja: simplicidad.

CDR — Common Data Representation (CORBA 2.0)

- Permite representar tanto big-endian como little-endian.
- Los valores se transmiten en el orden del emisor; se envía una etiqueta de arquitectura.
- No se envían etiquetas de tipo (acordadas previamente entre emisor y receptor).
- Más complejo que XDR, pero evita conversiones innecesarias.

XML

- Representación textual; muy legible e informativa.
- Ocupa mucho más espacio y el procesamiento es más lento.
- El código para codificar/decodificar es más complejo.

ASN.1 — Abstract Syntax Notation 1

- Cada valor se codifica con 4 campos: identificador, longitud, datos, indicador de fin.
- Usado con SNMP.
- Menos eficiente en espacio y procesamiento que XDR y CDR.

Protobuf — Protocol Buffers (Google)

- Formato binario de serialización de datos.
- Almacena datos estructurados de forma eficiente y compacta.

- Compatible con múltiples lenguajes e independiente de la plataforma.

Serialización de Objetos Java

- Específico para el lenguaje Java.

Comparación resumida

Formato	Codificación	Etiquetas de tipo	Endian	Uso principal
XDR	Binaria	No	Big-endian fijo	Sun RPC
CDR	Binaria	No	Configurable	CORBA
ASN.1	Binaria	Sí	Variable	SNMP
XML	Textual	Sí (tags)	N/A	Intercambio web (legacy)
JSON	Textual	Implícitas	N/A	APIs web modernas
Protobuf	Binaria	No (esquema externo)	N/A	APIs de alta performance

7. JSON — JavaScript Object Notation

Características Generales

JSON es un formato estándar abierto, ligero y basado en texto para el intercambio de datos legible por humanos. Fue especificado en 2000 por Douglas Crockford, estandarizado como ECMA-404 en 2013 y publicado como RFC 8259 en 2017.

- Es independiente del lenguaje de programación.
- Compatible con casi todo tipo de lenguaje, framework y biblioteca.
- Más pequeño, rápido y fácil de analizar que XML.
- Soporta estructuras de datos como objetos y matrices.
- No tiene métodos; es solo un formato de datos.

Tipos de Datos JSON

Tipo	Descripción	Ejemplo
String	Cadena de texto en comillas dobles	`"nombre"`, `"1234"`
Number	Valor numérico	`121`, `899`
Boolean	Verdadero o falso	`true`, `false`
Null	Valor vacío	`null`

Object	Par clave-valor entre `{}`	`{"nombre": "Juan"}`
Array	Lista de valores entre `[]`	`["a", "b", "c"]`

Objetos y Arrays

- **Objetos**: delimitados por `{}`, contienen pares clave-valor separados por comas. La clave debe ser un string y el valor cualquier tipo JSON válido.
- **Arrays (Matrices)**: delimitados por `[]`, contienen listas de valores de cualquier tipo JSON válido.

Uso en JavaScript

- `JSON.parse()`: convierte una cadena JSON en un objeto JavaScript.
- `JSON.stringify()`: convierte un objeto JavaScript en una cadena JSON.

Consideraciones importantes

- Las cadenas y nombres de propiedades **deben** usar comillas dobles (las simples no son válidas).
- Una coma o dos puntos mal ubicados pueden invalidar el archivo JSON.
- Se puede validar con herramientas como **JSONLint**.

Comparación JSON vs Protobuf

Característica	Protobuf	JSON
Desarrollador	Google	Douglas Crockford
Formato binario	Sí	No
Estandarización	No	Sí (ECMA-404)
Legible por humanos	Parcialmente	Sí
Comunidad	Pequeña	Muy grande

8. RPC — Llamada a Procedimiento Remoto

Introducción

RPC (*Remote Procedure Call*) es un mecanismo que permite a un procedimiento en la máquina A llamar a un procedimiento en la máquina B. La máquina A se bloquea hasta que el procedimiento de B termina. El objetivo es que la llamada remota sea lo más transparente posible: el programador no debe preocuparse por los mensajes enviados entre A y B.

La forma general de una llamada RPC es:

return_code = procedure_name(input_parameters, output_parameters)

Al terminar, el código de retorno indica si el procedimiento finalizó con éxito.

Componentes de RPC

- Uno o varios programas que se ejecutan en un servidor.
- Cada programa puede ejecutar varios procedimientos.
- Cada procedimiento realiza una función bien definida (por ejemplo, leer un archivo).
- Cada programa y cada procedimiento tienen un **identificador numérico** asignado.

Dificultades de RPC

- **Espacios de direcciones distintos**: los punteros y variables globales no tienen el mismo significado en cliente y servidor.
- **Tipos de datos**: no siempre son iguales en ambas máquinas; requieren codificación/decodificación a un formato prefijado.
- **Fallos**: cualquiera de las dos máquinas puede caer durante la comunicación.
- La transferencia de punteros se resuelve mediante **copia de zonas de memoria**, aunque en ciertos casos es más compleja.

Pasos de una llamada RPC (flujo completo)

1. El procedimiento cliente llama al **cliente-stub**.
2. El cliente-stub construye un mensaje (empaqueta / marshalling) y produce un system call.
3. El kernel del cliente envía el mensaje al kernel remoto (del servidor).
4. El kernel remoto entrega el mensaje al **server-stub**.
5. El server-stub desempaqueta el mensaje (unmarshalling) y llama al servidor.
6. El servidor ejecuta el procedimiento y devuelve los resultados al server-stub.
7. El server-stub construye un mensaje (empaqueta) y produce un system call.
8. El kernel del servidor envía el mensaje al kernel del cliente.
9. El kernel del cliente entrega el mensaje al cliente-stub.
10. El cliente-stub desempaqueta los resultados y los retorna al cliente.

Paso de parámetros y copia-restauración

En C, los escalares se pasan por valor y los arreglos por referencia. Como el espacio de direcciones es distinto en cliente y servidor, se usa la técnica de **copia-restauración**: los datos se copian del cliente al servidor en la llamada, y si deben retornarse, se copian de vuelta al cliente desde el servidor.

Modo sincrónico y asincrónico

- **Sincrónico**: el cliente queda bloqueado hasta que el servidor devuelve la respuesta. Es el modelo tradicional.
- **Asincrónico**: cuando no hay resultado que devolver (transferencia bancaria, agregar entradas a una base de datos, procesamiento batch, inicio de servicios remotos), no es eficiente que el cliente quede bloqueado.

9. Gestor de Puertos RPC

Muchos servidores RPC usan **puertos efímeros** (no conocidos de antemano). Para resolver este problema, cada host dispone de un programa llamado **gestor de puertos** (*portmapper*, o *rpcbind* en versiones más nuevas), que escucha en el puerto 111.

El gestor de puertos mantiene una tabla con:

- Los programas RPC locales activos.
- El número de versión de cada programa.
- El protocolo de transporte utilizado.
- El número de puerto en que se ejecuta cada programa.

Flujo de consulta: el cliente consulta al gestor de puertos en el puerto 111 preguntando dónde está el servicio N; el gestor responde con el puerto correspondiente; el cliente se conecta directamente al servicio.

10. Compiladores y Sun RPC

Compiladores RPC

Para generar aplicaciones RPC se usan compiladores como **Sun RPC (rpcgen)** o el paquete RPC de OSF. A partir de una definición de procedimiento remoto escrita en lenguaje RPC, el compilador genera automáticamente:

- Archivos de encabezamiento para cliente y servidor.
- Stub del cliente y del servidor.
- Rutinas XDR de codificación y decodificación.

Sun RPC

- Disponible como parte de Unix/Sun y en instalaciones NFS.
- Permite usar **UDP o TCP** como protocolo de transporte.
- Los mensajes están restringidos en longitud (teórico 64 KB, en la práctica 8 KB).
- Soporta autenticación: estilo Unix, propio de RPC, clave compartida o Kerberos.
- Modela las acciones como **idempotentes** con semántica **mínimo una vez**; requiere adaptaciones para suprimir duplicados y proveer semántica exactamente una vez.
- El procedimiento remoto solo puede aceptar **un parámetro** (se resuelve usando una estructura como parámetro).

11. XDR — External Data Representation

XDR es una representación estándar de datos que asegura la portabilidad entre distintas arquitecturas, liberando al programador de preocuparse por las diferencias de representación.

Incluye:

- Un **lenguaje de definición de estructuras de datos** inspirado en C (similar a ASN.1).
- Un conjunto de **reglas de codificación** más simples y compactas que ASN.1.

Características:

- Transferencia en palabras de **4 octetos en orden big-endian**.
- Tipos de datos: constantes, enteros, reales, opacos, cadenas, arreglos, estructuras, uniones y definiciones de tipo.
- A partir de tipos primitivos se construyen tipos complejos.
- Los **datos opacos** no son interpretados (útiles para datos encriptados).

12. Tratamiento de Fallos en RPC

Caída del Servidor

Cuando el servidor cae, el comportamiento depende del tipo de operación:

- Si la operación es **idempotente**, puede repetirse sin problemas (semántica *al menos una vez*).
- Si no es idempotente, se requiere la semántica *exactamente una vez*, que no está garantizada en todos los sistemas.

Caída del Cliente

Cuando el cliente cae durante la ejecución de un procedimiento remoto, el servidor queda ejecutando trabajo para un cliente que ya no existe. Estos procesos sin dueño se denominan **huérfanos** y el servidor debe tener una estrategia para resolverlos.

13. Servicios y Plataformas RPC

Servicios clásicos implementados con RPC y XDR

- **NFS** (Network File System)
- **Mount** (montaje de sistemas de archivos remotos)
- **Network Lock Manager**: previene que múltiples procesos modifiquen el mismo archivo simultáneamente.
- **Network Status Monitor**: reúne información de estado de otros nodos de la red.
- **NIS** (Network Information System): administración de red, crea y mantiene una base de datos de passwords y direcciones.
- **Portmapper / rpcbind**
- **Rusers** (servicio de difusión)

Plataformas modernas para desarrollo RPC

- **RPyC** (Remote Python Call): biblioteca Python para RPC y computación distribuida. Transparente, simétrica, sin necesidad de lenguajes de definición. Soporta comunicación sincrónica y asincrónica.
- **XML-RPC**: protocolo RPC que usa XML para codificar las llamadas y HTTP como mecanismo de transporte.
- **JSON-RPC**: protocolo RPC codificado en JSON. Muy simple, similar a XML-RPC. Permite notificaciones y múltiples llamadas que pueden responderse fuera de orden.
- **Thrift** (Facebook/Apache): framework para servicios escalables y multi-lenguaje. Combina un stack de software con un motor de generación de código. Compatible con C++, Java, Python, PHP, Ruby, Erlang, Perl, Haskell, C#, JavaScript, Node.js, entre otros.
- **Ice** (ZeroC): framework RPC de código abierto. Provee SDKs para C++, C#, Java, JavaScript, Objective-C, PHP, Python y Ruby. Ejecuta en Linux, Windows, OS X y Android.

Conclusiones sobre RPC

RPC es uno de los varios modelos para implementar comunicación distribuida. Es especialmente útil para soportar de forma transparente aplicaciones de tipo solicitud/respuesta. Sin embargo, no es apropiado para todas las aplicaciones debido a su overhead de rendimiento y su falta de flexibilidad. Antes de elegir un modelo de comunicación, es fundamental analizar los requisitos distribuidos de la aplicación.

14. Contenerización y Microservicios

Arquitectura Monolítica vs. Microservicios

Arquitectura monolítica: todos los procesos están estrechamente acoplados y se ejecutan como un único servicio. Si un proceso experimenta un pico de demanda, se debe escalar **toda** la arquitectura.

Arquitectura de microservicios: la aplicación se construye con componentes independientes donde cada proceso corre como un servicio separado. Cada servicio cumple una sola función y puede actualizarse, desplegarse y escalarse de forma independiente para satisfacer la demanda de esa función específica.

Contenedores

Un contenedor es una unidad ejecutable de software donde se empaqueta el código de aplicación junto con sus bibliotecas y dependencias, de modo que pueda ejecutarse en cualquier entorno. Sus características principales son:

- Son **ambientes de ejecución livianos** que proveen a las aplicaciones los archivos, variables y librerías que necesitan.
- Constituyen un mecanismo de **empaquetado lógico** que desvincula la aplicación del entorno en que se ejecuta.
- Facilitan el despliegue uniforme en centros de datos privados o en la nube.

Ventajas de los contenedores:

- Menos sobrecarga que las máquinas virtuales.
- Mayor portabilidad entre entornos.
- Funcionamiento independiente del entorno de ejecución.
- Facilitan el desarrollo y la adopción de DevOps.
- Permiten desplegar, replicar y escalar ambientes (desarrollo, prueba, producción) rápidamente.

Casos de uso:

- Implementación de microservicios.
- Adopción de prácticas DevOps.
- Entornos de nube híbrida o multi-cloud.
- Migración de aplicaciones monolíticas.

Contenedores vs. Máquinas Virtuales (VMs)

Aspecto	Máquina Virtual	Contenedor
SO huésped	Incluye SO completo	Solo la aplicación y sus dependencias
Peso	Pesado	Liviano
Velocidad de inicio	Lenta (minutos)	Rápida (segundos)
Portabilidad	Menor	Mayor
Aislamiento	Completo	Compartido a nivel de kernel

Los contenedores virtualizan el sistema operativo (normalmente Linux), por lo que cada contenedor contiene únicamente la aplicación y sus dependencias. La ausencia del SO huésped los hace más ligeros, rápidos y portátiles.

15. Docker

Docker es una plataforma de código abierto que permite a los desarrolladores crear, implementar, ejecutar, actualizar y gestionar aplicaciones dentro de contenedores ligeros y portátiles. Empaqueta el código y sus dependencias (librerías, configuraciones) para asegurar que la aplicación funcione igual en cualquier entorno.

> Docker es la herramienta de contenerización más utilizada, con una cuota de mercado del 82,84%, convirtiéndose en el estándar *de facto* del mercado.

Docker Engine

Es la tecnología cliente-servidor que permite construir y contenerizar aplicaciones. Sus componentes principales son:

Cliente (Docker Client)

- Es la interfaz de línea de comandos con la que el usuario interactúa.
- Comandos principales: `docker run`, `docker build`, `docker ps`.
- Se comunica con el Daemon de Docker.

Daemon (dockerd)

- Es el motor principal de Docker.
- Corre en segundo plano en el sistema.
- Se encarga de: crear y ejecutar contenedores, manejar imágenes, gestionar redes y volúmenes.